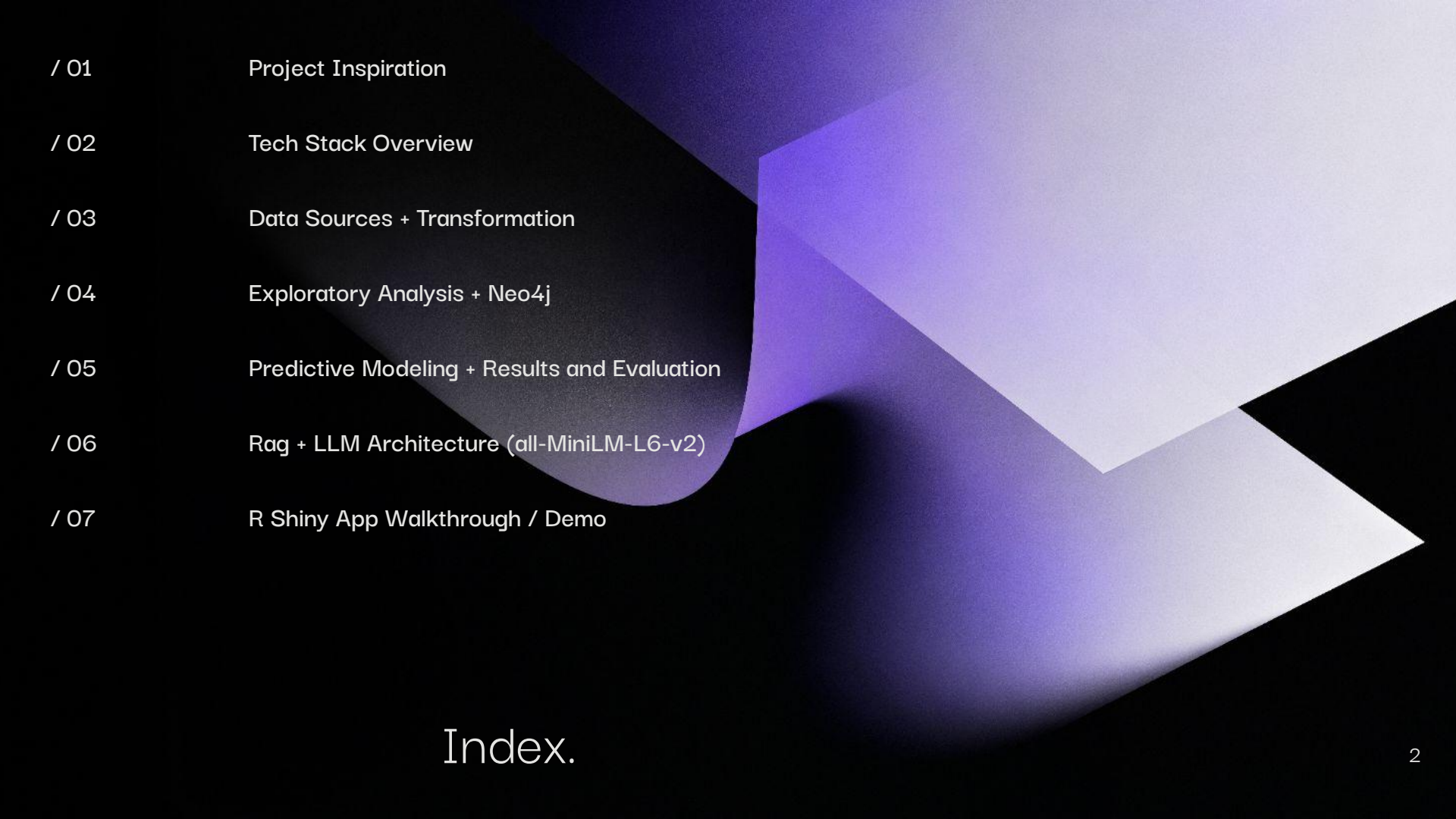


Nashville //DS9
Software
School

Grant Alan

NLP Email Optimization RAG + OpenAI + Mailchimp

- 
- / 01 Project Inspiration
 - / 02 Tech Stack Overview
 - / 03 Data Sources + Transformation
 - / 04 Exploratory Analysis + Neo4j
 - / 05 Predictive Modeling + Results and Evaluation
 - / 06 Rag + LLM Architecture (all-MiniLM-L6-v2)
 - / 07 R Shiny App Walkthrough / Demo

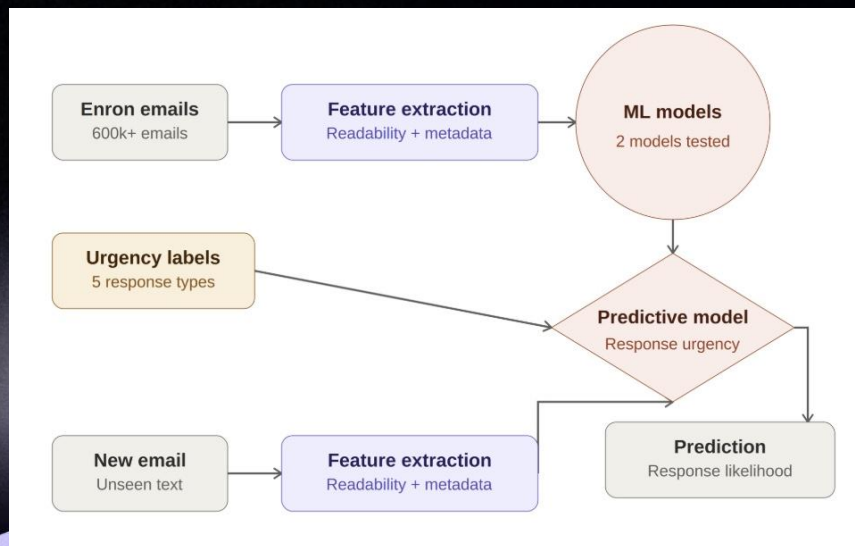
Index.

/ 01

Project Inspiration.



Idea & Inspiration



The idea for this project all began when I discovered natural language processing and text classification. I realized that in this future age of technology, human correspondence is quietly disappearing. I wanted to understand how AI is processing emails at scale and which words make an impact on response rate.

https://mediacenter.ibm.com/media/Text+Classification+AI+Techniques+and+Real-World+Applications/1_o3chonwn

Emails are no longer being processed by humans, but by machines that categorize text based on keywords. Certain morphemes signify urgency and automated systems are processing them at scale.

The goal of this project is to create a tool that predicts the likelihood of an email response based on text features alone. As a stretch goal I wanted to Infuse an LLM with real world marketing savvy to increase consumer impact.

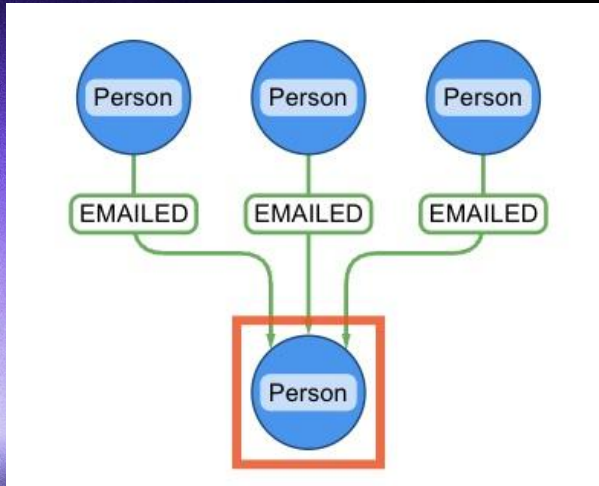
/ 02

Tech Stack



Technology and Libraries accessed.

<https://www.yworks.com/yed-live/>



/ 01 UI framework is compiled with R Shiny, bslib and plotly. Network Analysis Visualized in a Jupyter notebook using Neo4j (GraphWidget and GraphDatabase)

/ 02 ML and Backend development consists of R and Python code. Libraries include reticulate, scikit-learn, pandas, numpy, matplotlib and syllables.

/ 03 AI and Rag layer is composed of fastembed (all-MiniLM-L6-v2), OpenRouter (gpt-oss-20b), OpenAI Python client and Mailchimp knowledge base.

Data comes from Kaggle Enron email-reply dataset, Enron mass email corpus and 25 curated best practice email chunks available at Mailchimp.com

Note: Heatmap, ROC curve and all other visualizations are built with Matplotlib and/or Seaborn.

/ 03

Data Sources + Transformation



Enron Email-Reply Dataset.

<https://www.kaggle.com/datasets/oanannv/enron-email-reply-dataset>



/ 01 When Enron collapsed in 2001, federal investigators released over 600,000 internal company emails as public record. It is the largest public-domain email corpus that exists

/ 02 I chose this dataset so that I could model email response time. There are 400,000 real corporate emails that I use to uncover network structure, response latency, and hierarchy in this project

/ 03 Between a clean Kaggle dataset and the original messy 600k email dump I was able to merge them and add features like response time, a response indicator and various text features to predict response time.

Text Features include 'char_count', 'exclamation_count', 'token_count', 'avg_word_length', 'question_mark', 'stop_word_count', 'syllable_count'

Mailchimp Knowledge Base & Best Practice.

<https://mailchimp.com/help/best-practices-mailchimp-email/>



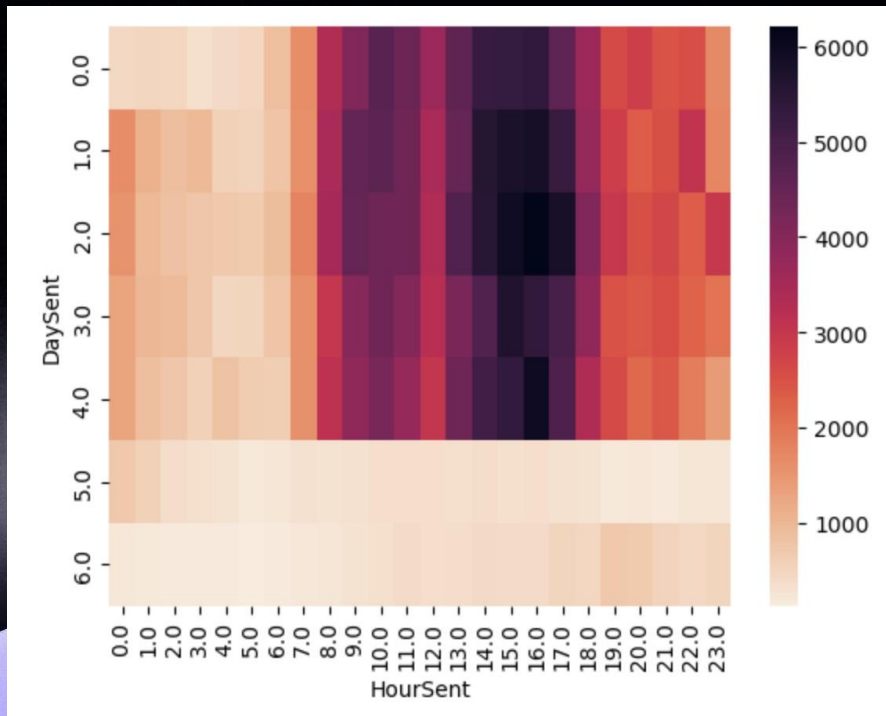
The Mailchimp knowledge base is 25 email marketing best-practice tips covering subject lines, tone, urgency and send times these are embedded with fastembedding. When a user submits an email, the system finds the 5 most relevant tips for that specific email and sends them to the LLM along with the email's predicted response score, so the AI rewrites the email using real, research-backed guidance instead of generic suggestions. The Rag system uses the response score infused with knowledge from Mailchimp to help prompt the **openai/gpt-oss-20b:free model**.

- Subject Lines
- Email Length and Structure
- Tone and Clarity
- Urgency and Engagement
- Mobile and Formatting
- Sending Time

/ 04

Exploratory Analysis + Neo4j

Exploratory Analysis.



A couple factors I wanted to explore are the spread of emails across time and the volume at which they come in. I divided the emails into different urgency buckets to indicate which emails are getting quick responses and which ones are being ghosted. The emails are broken up into 5 buckets. These variables are used later on when establishing a target variable in the neural network model.

ghosted	344643
same_day	8677
instant	2977
fast	2346
next_day	1108

This heat map answers my first question. What is the spread of emails across time within the enron corpus? Peak activity is between 2-4pm with a very strong volume of emails coming in Tuesday and Wednesday. There is a burst Friday around 4pm - It's probably people tying up loose ends before the weekend.

Email volume reflects typical business behavior minimum and flat rate or emails coming in.

1998-1999:
baseline
period.

Gradual climb of email volume which is line with business development and expansion.

2000:
steady
growth.

Larger spikes and dips in email volume due to internal financial struggle.

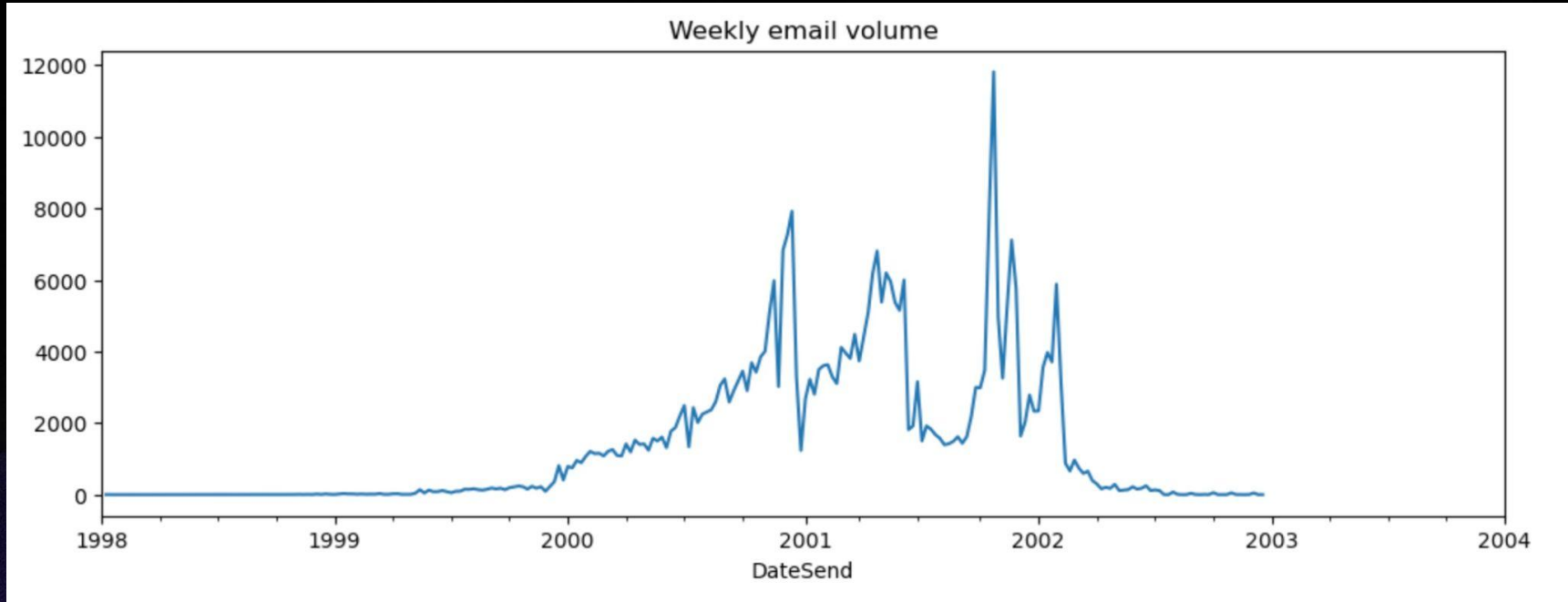
Late 2000 -
early 2001:
growing
volatility

The tallest peak hitting nearly 12,000 emails. Merger canceled, Huge cut in shareholder equity.

Fall 2001:
the big
spike

History of Enron Email Corpus.

Enron weekly email volume between 1989-2003



Neo4j Graph Data & Network Analysis.

[9]:	avg_response	n_replies
From		
Maggi, Mike	1.851064	47
ted.evans@enron.com	2.600000	5
Rybarski, Amanda	4.000000	10
chance.rabon@enron.com	5.200000	5
tom.moran@enron.com	6.200000	5
mjillard@beaconelectric.com	8.714286	7
davis.thames@enron.com	9.666667	6
john.knock@elpaso.com	10.000000	5
llmaser@degeurinrealty.com	10.000000	5
joseph.deffner@enron.com	10.800000	5
ek@a-klaw.com	12.000000	6
Valdes, John	12.666667	6
Sever, Stephanie	13.800000	5
nmann@erac.com	13.800000	10
luis.mena@enron.com	15.000000	7
mike.maggi@enron.com	15.129464	448
alexandra.villarreal@enron.com	15.888889	45
ken.lewchuk@enron.com	16.200000	5
doug.sewell@enron.com	16.428571	7
chris.booth@enron.com	16.428571	7

/ 01

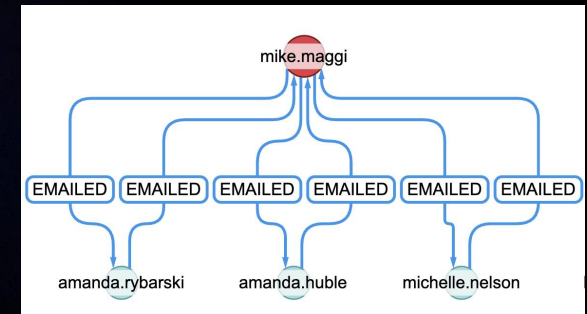
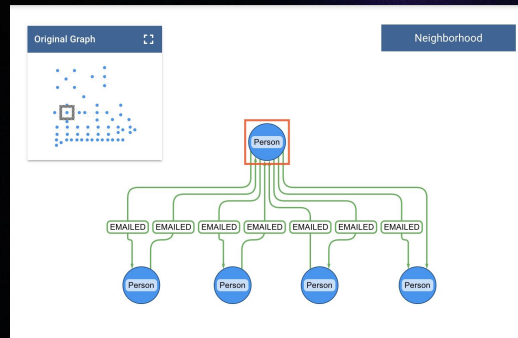
Text features alone can only tell so much about response time. What other factors predate response time? Could position inside of a business influence response time?

/ 02

A large factor at play is political hierarchy within a business. These discrepancies can vastly affect the accuracy of a predictive model especially in this case where our target is response time.

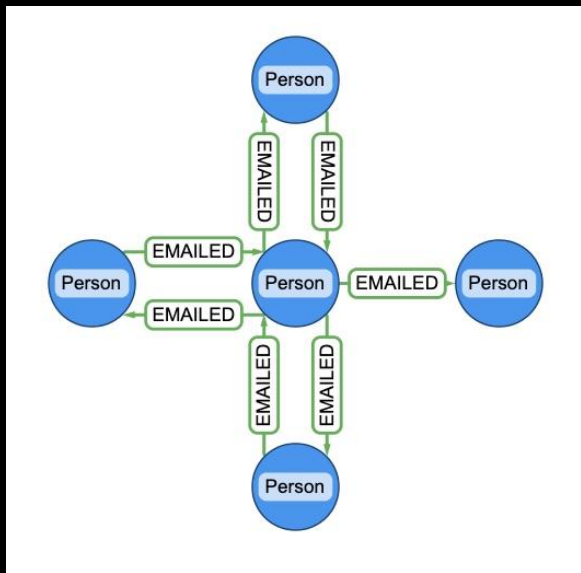
/ 03

Here we visualize which entities within the company are getting the quickest response and which people are receiving the largest volume of emails.



Enron Neo4j Graph Database

<https://www.yworks.com/yed-live/>



/ 05

Predictive Modeling - Results & Evaluation

Logistic Regression

/ 01

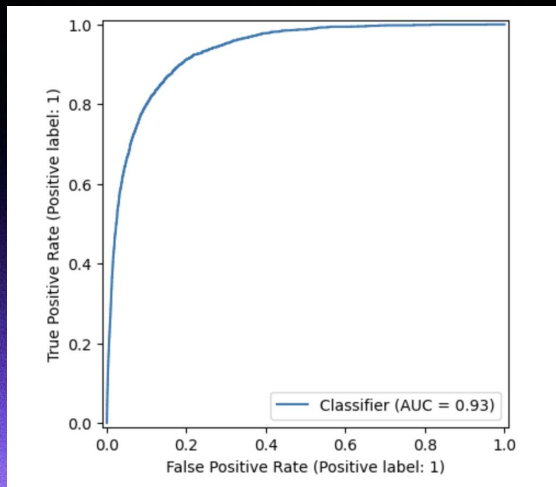
The Logistic Regression model is trained on various text features extracted from the original enron email dataset. These functions are then recycled to extract metadata from text input within my R application

/ 02

The model is trained on 'char_count', 'exclamation_count', 'token_count', 'avg_word_length', 'stop_word_count', 'syllable_count', 'has_question', 'email_send'. These variables are used to predict the likelihood of an email response within 3 hours.

/ 03

Our model has 93% AUC, good at telling the two groups apart overall. Because our data is heavily skewed (only 3% of emails are in the class we care about), it plays it safe and misses most of them, catching just 17% of actual positive cases. Next step would be to tune the threshold and rebalance the classes.

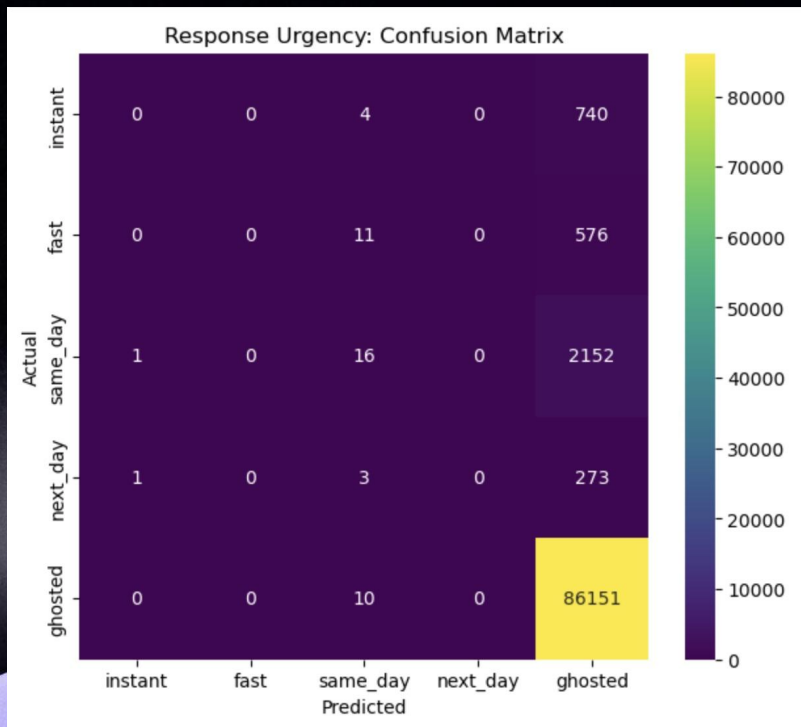


	precision	recall	f1-score	support
0	0.97	1.00	0.98	69690
1	0.56	0.17	0.27	2261
accuracy			0.97	71951
macro avg	0.77	0.59	0.63	71951
weighted avg	0.96	0.97	0.96	71951

```
[44]: # Confusion matrix
      print(confusion_matrix(y_test, y_pred))

      [[69376  314]
       [ 1866  395]]
```

Neural Network Model



I trained a multi-layer perceptron (MLPClassifier) to predict how urgently an email would be answered and bucketed into five classes: instant, fast, same_day, next_day, or ghosted. The model achieves 96% overall accuracy, but this is misleading: the "ghosted" class makes up 96% of the data, so the model essentially learned to predict "ghosted" for nearly every email and ignored the other four categories entirely. Each of which scored 0.00 precision/recall/F1.

This is a severe class imbalance problem, and the next step is class rebalancing (weighting or resampling) so the model can actually learn to distinguish fast responses from slow ones, rather than defaulting to the majority class.

The confusion matrix heatmap demonstrates a model collapsing into a majority prediction due to severe class imbalance as opposed to a clear diagonal line indicating accuracy.

/ 06

RAG + LLM Architecture (all-MiniLM-L6-v2)

Rag + LLM + Mailchimp Fastembed



Gpt-oss-20: Free

Low level AI tool with no instructions or training on what ACTUALLY gets responses inside of a legitimate corporate environment. Guessing the right conditions for a quick response without context.

/ BEFORE

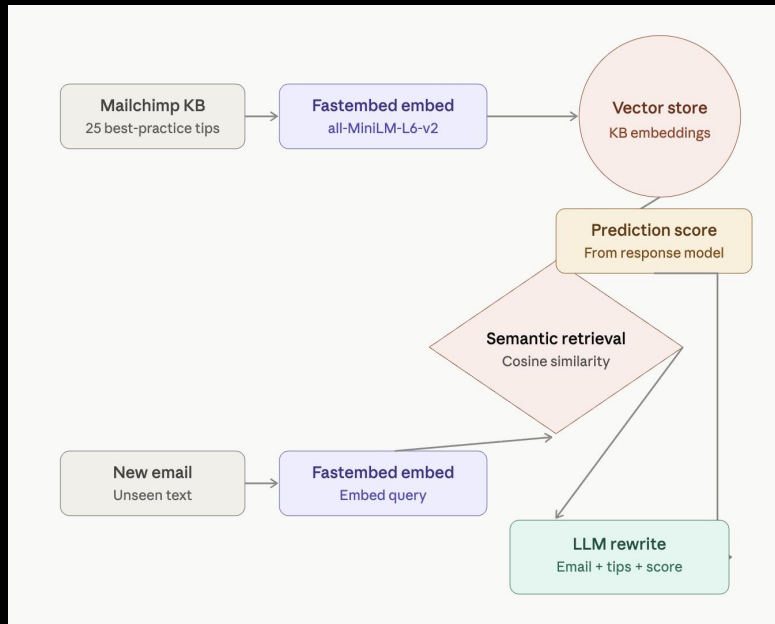
/ AFTER



RAG-powered LLM

Fastembed retrieves the most relevant Mailchimp best practice for each email, then an LLM uses them plus the model's response likelihood score to instantly rewrite the email and explain what changed.

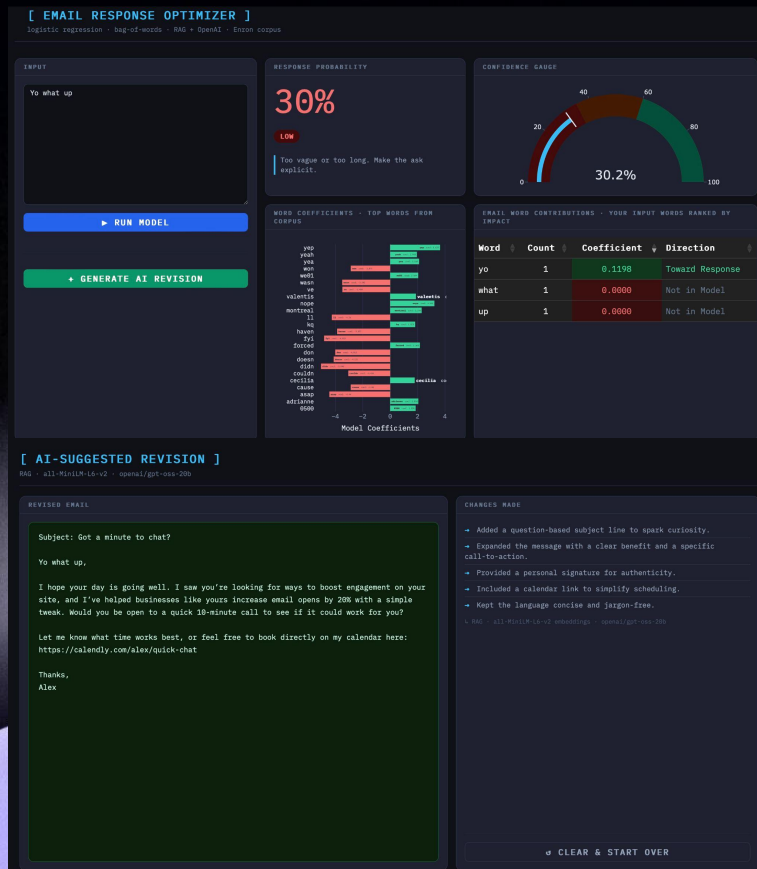
RAG Architecture Flow Chart



/ 07

R Shiny Walkthrough & Demo

Email Response Optimizer

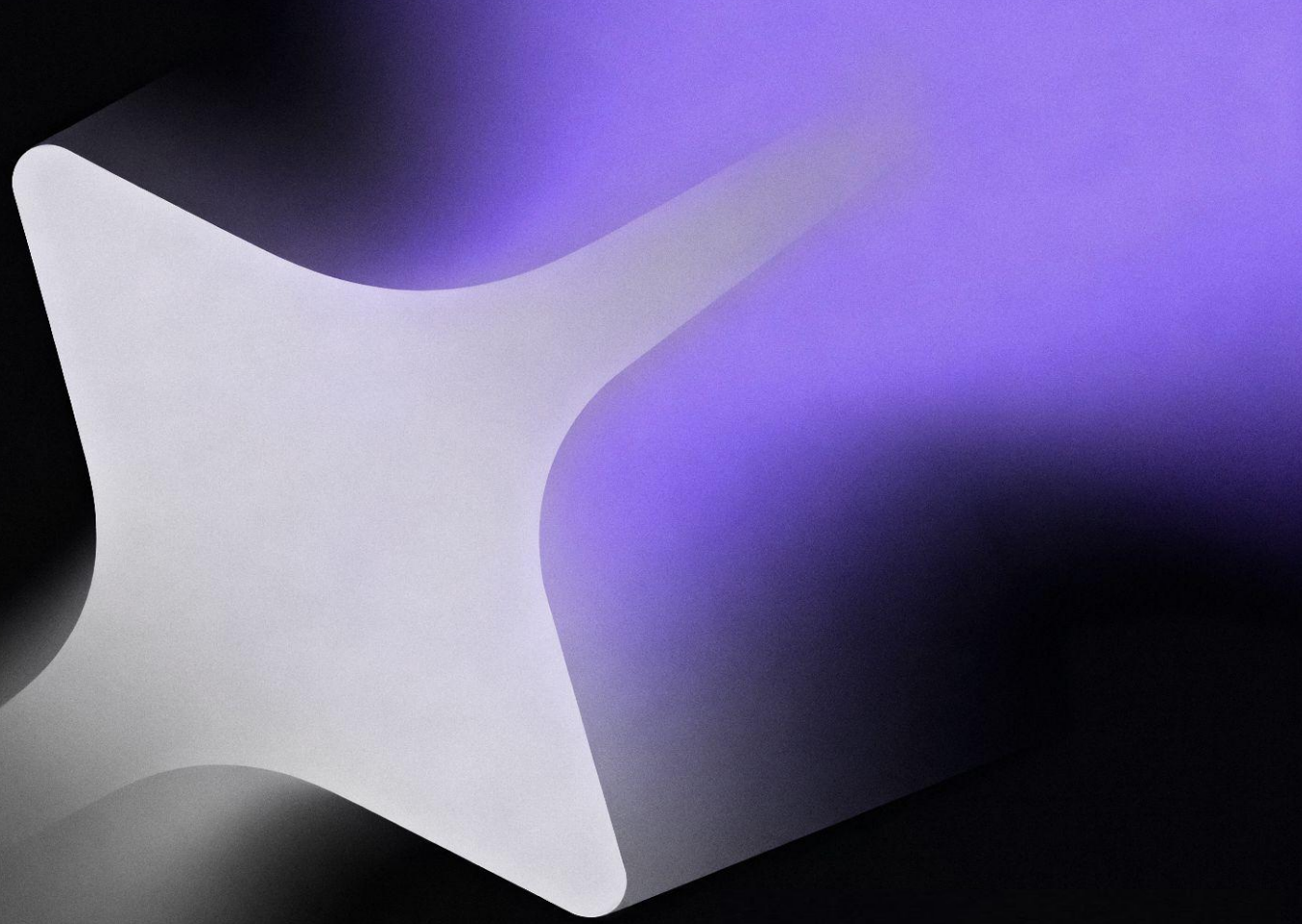


Inside the R shiny Web application you can type in a sample email and get a predicted probability of a response within 3 hours. The UI also shares which words are contributing to getting a quick response within the model. It also shares which phrases in your sample email are pushing your email in the direction of a quick response or further away. The RAG and AI model will create revised emails for you once you click the “Generate AI Revision Button”. The app is live and deployed on R shiny’s website.

The original platform was developed using the owl-alpha model from openrouter. It has since depreciated and I have moved to a free tier gpt-oss model. The application is simultaneously running Python code in R via “reticulate”. In the future I plan to access the model helper function via Fast API to avoid excess memory pooling and increase processing speed.

speeds. Enjoy!

<https://grantalan.shinyapps.io/email-response-optimizer/>



Thanks.